

# Cours de Programmation en C

## Travaux Pratiques

### — TP8 —

On se propose de représenter des expressions algébriques au moyen d'un arbre de syntaxe (*Abstract Syntax Tree*<sup>1</sup>). Un arbre ressemble quelque peu à une liste chaînée, sauf que chaque élément (appelé *nœud*) a deux successeurs (appelés *fil*, par analogie avec l'arbre généalogique. On les appelle *fil gauche* et *fil droit*). Les nœuds terminaux (qui ne possèdent pas de fils) sont appelés des *feuilles* (toujours par analogie à l'arbre).

Un exemple d'expression algébrique représentée par un arbre est donné à la figure 1. Les nœuds peuvent contenir une chaîne de caractères (" $+$ ", " $\sin$ ", " $x$ ") ou bien des valeurs réelles (uniquement pour les feuilles). Par convention, nous décidons qu'ils contiendront une chaîne de caractères et un réel, et que la chaîne sera vide (c'est-à-dire un pointeur égal à `NULL`) s'il faut considérer le réel. Les pointeurs vers les fils gauche et/ou droit seront égaux à `NULL` s'il n'y a pas de fils gauche et/ou droit (cas des feuilles ou bien de fonctions telles que *sinus* qui n'ont qu'une seule opérande, et que l'on mettra sur le fils gauche).

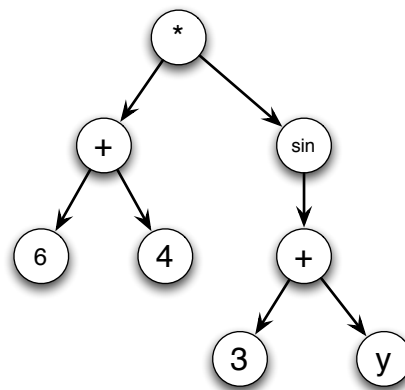


Figure 1: Arbre syntaxique représentant l'équation  $(6 + 4) * \sin(3 + y)$

## 1 Construction d'arbres binaires

### Question 1

Dans un fichier `arbre.h`, écrire une structure de donnée auto-référencée `t_noeud` permettant de décrire un nœud.

### Question 2

Écrire dans `arbre.c` une fonction permettant de créer un nœud à partir d'une valeur, d'une chaîne de caractères, d'un fils gauche et d'un fils droit. Cette fonction aura pour prototype:

```
t_noeud* creerNoeud( float val, char* ch, t_noeud* filsG, t_noeud* filsD);
```

N'oubliez pas d'allouer de la mémoire pour le nœud lui-même et pour la chaîne de caractères, qu'il faudra recopier.

### Question 3

En déduire les fonctions `creerNoeudOp`, `creerFeuilleReel` et `creerFeuilleVar` qui permettent de créer un nœud avec un opérateur, une feuille avec un réel et une feuille avec une variable, respectivement. Elles appelleront juste la fonction `creerNoeud` avec les bons paramètres. Elles devront avoir les prototypes suivants:

```
t_noeud* creerNoeudOp( t_noeud* filsG, char* ch, t_noeud* filsD);
t_noeud* creerFeuilleReel( float val);
t_noeud* creerFeuilleVar( char* ch);
```

<sup>1</sup>[http://en.wikipedia.org/wiki/Abstract\\_syntax\\_tree](http://en.wikipedia.org/wiki/Abstract_syntax_tree)

**Question 4**

Écrire une procédure `detruireArbre` permettant de détruire **récurivement** un arbre (donc détruit ses deux fils (s'ils existent) en s'appelant puis désalloue ce qu'il faut pour le nœud lui-même).

**Question 5**

Écrire une fonction `nbFils` permettant de compter le nombre de fils d'un nœud.

**Question 6**

On considère le code suivant :

```
t_noeud* arbre = creerNoeudOp( creerFeuilleReel(12), "+", creerNoeudOp(
    creerFeuilleReel(4), "*", creerFeuilleVar("x") ) );
```

Il permet de créer un arbre pour l'expression  $12+4*x$ . Dans un fichier `mainP1.c`, utiliser cette expression pour tester les fonctions précédemment écrites. Créer aussi un arbre pour l'expression  $(12+x) * (6/25)^2$ .

**2 Saisie et affichage d'une expression**

On a vu, la création d'un arbre n'est pas une chose aisée. Pour faciliter la saisie, on vous fournit une fonction `lectureInfixee` qui permet de créer un arbre à partir d'une expression *infixée* (c'est-à-dire sous forme classique, où les opérandes sont de chaque côté de l'opérateur, comme dans l'expression  $12+4*x$ ). On ne considère ici que les opérations  $+$ ,  $-$ ,  $*$ ,  $/$ ,  $^$  et `sin`. Il faut le récupérer dans `/home/sasl/shared/ei-ii/C/TP7/`.

**Question 7**

Écrire, dans un fichier `mainP2.c`, un programme saisissant une chaîne de caractère et construisant l'arbre correspondant (attention, `scanf` s'arrête au 1<sup>er</sup> espace, donc pas il ne faut pas taper d'espace dans la formule ou bien il faut utiliser une autre méthode de saisie, avec `getchar` par exemple).

On cherche maintenant à afficher un arbre. On mettra les fonctions correspondantes dans `equation.c`.

**Question 8**

On commence par l'écriture *post-fixée* (les opérandes précèdent l'opérateur, comme dans  $12\ 4\ x\ *\ +$ ). Écrire une procédure **réursive** qui affiche un arbre en notation post-fixée. L'utiliser dans `mainP2.c`.

**Question 9**

Écrire une procédure réursive qui affiche un arbre en notation *infixée*. Attention à bien parenthéser. Pour plus de facilité, on pourra distinguer les cas suivant le nombre de fils d'un nœud. L'utiliser dans `mainP2.c`.

**3 Évaluation d'une expression**

On cherche maintenant à évaluer (calculer) une expression. Les branches comprenant des variables ne pourront, par contre, pas être calculées (on ne cherchera pas ici à remplacer une variable par une valeur donnée). Le but des questions qui suivent est d'écrire une fonction `int evaluer(t_noeud* arbre)` qui renvoie un entier indiquant si l'arbre a pu être évalué ou non. Si l'arbre peut être évalué, la fonction le modifiera en place pour obtenir un arbre composé d'une unique feuille contenant le résultat de l'opération stockée au départ dans l'arbre.

**Question 10**

Dans le fichier `equation.c` écrire la fonction `evaluer` qui ne gèrera pour le moment que le cas des feuilles.

**Question 11**

Écrire une procédure `void remplacerNoeud(t_noeud* arbre, float val)` qui modifie l'arbre passé en paramètre pour le remplacer par une feuille contenant la valeur passée en second paramètre. Vous prendrez soin de désallouer tous les éléments de l'arbre à supprimer.

**Question 12**

À l'aide de la procédure `remplacerNoeud`, modifier la fonction `evaluer` pour qu'elle gère le cas des opérateurs unaires (ici seulement `sin`). Vous penserez à vérifier que l'arbre passé en paramètre a bien la structure d'une opération unaire puis à évaluer le(s) sous arbre(s) nécessaire(s) avant de modifier l'arbre.

**Question 13**

Ajouter la gestion des opérateurs binaires. Dans un premier temps on ne traitera que l'opération  $+$ . Puis lorsque vous aurez testé votre fonction, la compléter pour gérer les autres opérateurs binaires ( $-$ ,  $*$ ,  $/$ ,  $^$ ). On prendra soin de

vérifier que l'on n'effectue pas de division par 0 et que les exposants sont des entiers (vous pourrez par exemple pour cela utiliser la fonction `round` qui arrondit un réel à l'entier le plus proche).

### Question 14

Dans le fichier `mainp3.c` écrivez un programme qui lit des opérations au clavier jusqu'à l'entrée de la chaîne "EXIT" et pour chaque opération affiche l'arbre avant et après l'évaluation.

## 4 Bonus : la dérivation

On cherche maintenant à faire de la dérivation formelle. Le but des questions est d'écrire une procédure `void derivier(t_noeud* arbre, char* var)` qui procède à la dérivation de l'arbre par rapport à la variable donnée par la chaîne `var`.

### Question 15

On commence par écrire les règles de dérivation pour les constantes et les variables (on supposera que les variables sont indépendantes entre elles, et que donc  $\frac{\partial y}{\partial x} = 0$ ).

### Question 16

Écrire la règle de dérivation de l'opérateur `+` (si  $f$  et  $g$  sont des expressions, alors  $\frac{\partial(f+g)}{\partial x} = \frac{\partial f}{\partial x} + \frac{\partial g}{\partial x}$ ).

### Question 17

Écrire une fonction `t_noeud* dupliquer(t_noeud* arbre)` qui duplique l'arbre passé en paramètre (copie récursive de cet arbre et de ses fils) et renvoie le nouvel arbre créé.

### Question 18

Écrire la règle de dérivation du `*` (si  $f$  et  $g$  sont des expressions  $\frac{\partial(f*g)}{\partial x} = \frac{\partial f}{\partial x} * g + f * \frac{\partial g}{\partial x}$ ).

### Question 19

Écrire la règle de dérivation du sinus ( $\frac{\partial \sin(f)}{\partial x} = \frac{\partial f}{\partial x} * \cos(f)$ ).

### Question 20

Écrire la règle du `^` (on se restreindra au cas où l'opérande de droite est une constante :  $\frac{\partial f^k}{\partial x} = k * \frac{\partial f}{\partial x} * f^{k-1}$ ).

## 5 Bonus : la simplification

Une fois que l'on a effectué une dérivation, il faut aussi souvent simplifier l'expression. Par exemple, le fait de dériver par rapport à  $x$  l'expression  $6*x$  donne pour résultat  $6*1 + 0*x$ , qu'il faut bien entendu ensuite simplifier par 6. On va donc écrire une fonction `t_noeud* simplifier(t_noeud* arbre)` ou `void simplifier(t_noeud** pArbre)` qui simplifiera un arbre (le nouvel arbre sera renvoyé, ou bien modifié par le passage par adresse d'un `t_element*`).

### Question 21

Écrire le code correspondant à la simplification de  $0*f$  et  $f*0$  (où  $f$  est un arbre quelconque).

### Question 22

Écrire le code correspondant à la simplification de  $1*f$  et  $f*1$ .

### Question 23

Écrire le code correspondant à la simplification de  $0+f$  et  $f+0$ .

### Question 24

Écrire le code correspondant à la simplification de  $f^0$ .

### Question 25

Écrire le code correspondant à la simplification de  $f^1$ .